

Universität Duisburg-Essen

Mechatroniklabor:
Entwicklung einer Messtechnik zur
dreidimensionalen Erfassung von
Pflanzenkoordinaten

WS 24/25

Professor: Prof. Dr.-Ing. Söffker

Betreuer: Mazen Zeno M.Sc.

Verfasser:

- Ali Elbaradie, [REDACTED]
- Hassan [REDACTED]
- Ismail [REDACTED], [REDACTED]
- Sahl [REDACTED], [REDACTED], [REDACTED]

Abgabe: 01.02.2025

Inhaltsverzeichnis

1. Einleitung.....	4
2. Stand der Technik.....	5
2.1 Maschinelles Lernen (Machine Learning) in MATLAB	5
2.2 Deep Learning in MATLAB	7
2.3 Image Processing and Computer Vision in MATLAB.....	9
2.4 Kamerakalibrierung	12
2.5 Visual SLAM mit RGB-D-Kameras.....	13
3. Hauptteil.....	15
3.1 Aufbau der Prüfstand.....	15
3.2 Fehlerdiagnose und Lösungsansätze	18
3.2 Entwicklung und Integration eines Gehäuses für die Micro-Limit-Switches	19
3.3 Beschreibung der Arduino-Code-Entwicklung	20
3.4 Bildverarbeitung	24
4. Ergebnisse und Auswertung.....	29
5. Anhang	31
Arduino-Code.....	31
Datenvorbereitung.....	54
Mask R-CNN for Instance Segmentation	55
Visual SLAM with RGB-D Camera	57

Abbildungsverzeichnis

Abbildung 1: Aufbau der Kamera.....	15
Abbildung 2: Aufbau des Prüfstands	16
Abbildung 3: Erweiterung des Prüfstands.....	17
Abbildung 4: 3D-Modell des Gehäuses	19
Abbildung 5: Der Limit-Switch im Gehäuse am Prüfstand	19
Abbildung 6: Graphical User Interface zur Steuerung der Kameraposition.....	23
Abbildung 7: Funktionsweise des Image Labelers.....	24
Abbildung 8: Bilder für die Kamerakalibrierung mit der Nutzung von Schachbrett....	27
Abbildung 9: Zwei Bilder der Pflanze mit zwei verschiedene Position	28
Abbildung 10: Merkmalsextraktion und -verfolgung (Links), 3D-Punktwolke der Szene (Rechts)	28
Abbildung 11: Das Ergebnis für die Erkennung der Pflanze	29

1. Einleitung

Das Fachgebiet der Mechatronik lässt sich als interdisziplinär bezeichnen, da es Inhalte der Mechanik, Elektronik, Informatik sowie der Steuerungstechnik miteinander vereint. Die Mechatronik erlaubt die Konzeption moderner Systeme und Geräte, die sich durch eine hohe Automatisierung und Präzision auszeichnen und vielseitig einsetzbar sind. Die Mechatronik spielt eine entscheidende Rolle in einer Vielzahl von Bereichen, von der Automobilindustrie über die Medizintechnik bis hin zur Landwirtschaft. Sie ist eine wesentliche Disziplin, wenn es darum geht, technologische Innovationen voranzutreiben und komplexe Herausforderungen zu bewältigen.

Ein zentrales Forschungsfeld der Mechatronik ist die Entwicklung moderner Messtechniken, die präzise und automatisierte Erfassungsmethoden für unterschiedlichste Anwendungen ermöglichen. Insbesondere in der Land- und Umwelttechnik gewinnen dreidimensionale Erfassungssysteme zunehmend an Bedeutung, da sie eine detaillierte Analyse räumlicher Strukturen erlauben und wertvolle Daten zur Optimierung von Anbau- und Umweltbedingungen liefern. Die zunehmenden globalen Herausforderungen, wie Klimawandel und nachhaltiger Ressourcennutzung, machen die Digitalisierung und Automatisierung landwirtschaftlicher Prozesse zu einer unabdingbaren Notwendigkeit. Präzise Messmethoden können einen wesentlichen Beitrag dazu leisten, die Erträge zu maximieren, den Einsatz von Düngemitteln und Wasser zu optimieren und gleichzeitig die ökologischen Auswirkungen zu minimieren.

Im Rahmen dieses Projekts wird ein innovatives Messsystem zur dreidimensionalen Erfassung von Pflanzenkoordinaten entwickelt. Ziel ist es, eine hochgenaue, automatisierte und effiziente Lösung bereitzustellen, die präzise Positionsdaten von Pflanzen erfassen kann. Die Kombination aus moderner Sensorik und künstlicher Intelligenz soll dabei eine zuverlässige Analyse ermöglichen und neue Erkenntnisse für den landwirtschaftlichen und umwelttechnischen Bereich liefern. Die im Projekt gewonnenen Erkenntnisse können langfristig dazu beitragen, die Effizienz von Anbauprozessen zu steigern und nachhaltige landwirtschaftliche Strategien zu entwickeln.

2. Stand der Technik

2.1 Maschinelles Lernen (Machine Learning) in MATLAB

Maschinelles Lernen ist ein zentraler Bestandteil der künstlichen Intelligenz (KI), bei dem Algorithmen entwickelt werden, um aus Daten zu lernen und Vorhersagen oder Entscheidungen zu treffen, ohne dass diese explizit programmiert werden müssen. MATLAB bietet eine umfangreiche Sammlung von Werkzeugen und Funktionen, die die Implementierung von Machine-Learning-Modellen erheblich vereinfachen.

Typische Schritte im Machine-Learning-Prozess

- **Datensammlung**

Der erste Schritt im Machine-Learning-Prozess ist die Erfassung von Daten. Diese Daten können aus verschiedenen Quellen stammen, wie beispielsweise Sensoren, Bildern, Texten oder Messungen. Ein Beispiel wäre das Sammeln von Bildern verschiedener Pflanzenarten, um ein Modell zur Klassifikation von Pflanzen zu trainieren.

- **Vorverarbeitung**

Nach der Datensammlung müssen die Daten bereinigt und normalisiert werden, um sie für die Analyse vorzubereiten. Dies kann das Entfernen von Ausreißern, das Füllen fehlender Werte oder die Skalierung der Daten umfassen. Die Qualität der Vorverarbeitung hat einen erheblichen Einfluss auf die Leistung des Modells.

- **Feature-Extraktion**

In diesem Schritt werden relevante Merkmale aus den Daten identifiziert, die für das Modell von Bedeutung sind. Bei Bildern von Pflanzen könnten dies beispielsweise die Länge, Form oder Farbe der Blätter sein. Die Auswahl der richtigen Merkmale ist entscheidend, um die Genauigkeit des Modells zu verbessern.

- **Modelltraining**

Mit den vorverarbeiteten Daten und extrahierten Merkmalen wird ein Machine-Learning-Modell trainiert. Dabei lernt der Algorithmus Muster und Zusammenhänge in den Daten, um zukünftige Vorhersagen zu treffen. MATLAB bietet eine Vielzahl von Algorithmen, die für diesen Schritt verwendet werden können.

- **Validierung und Testen**

Nach dem Training wird das Modell mit Testdaten bewertet, um seine Genauigkeit und Robustheit zu überprüfen. Dies hilft sicherzustellen, dass das Modell nicht nur die Trainingsdaten gut kennt, sondern auch auf neuen, unbekannten Daten gut abschneidet.

- **Anwendung**

Sobald das Modell trainiert und validiert ist, kann es zur Vorhersage oder Klassifikation neuer Daten eingesetzt werden. Beispielsweise könnte das Modell verwendet werden, um neue Pflanzenbilder automatisch zu klassifizieren.

MATLAB-Werkzeuge für Maschinelles Lernen

MATLAB bietet die **Statistics and Machine Learning Toolbox**, die eine breite Palette von Algorithmen für überwachtes und unüberwachtes Lernen bereitstellt. Dazu gehören:

- **Überwachtes Lernen:**

- Lineare Regression
- Support Vector Machines (SVM)
- Entscheidungsbäume und Random Forests
- Neuronale Netze

- **Unüberwachtes Lernen:**

- k-means-Clustering
- Hauptkomponentenanalyse (PCA)
- Hierarchisches Clustering

Mit diesen Werkzeugen können Sie komplexe Machine-Learning-Modelle effizient implementieren und anpassen. MATLAB unterstützt auch die Integration mit anderen KI-Frameworks wie TensorFlow und PyTorch, wodurch die Möglichkeiten noch weiter erweitert werden.

Zusätzlich bietet MATLAB eine intuitive Benutzeroberfläche, die es ermöglicht, den gesamten Machine-Learning-Prozess visuell zu gestalten und zu analysieren. Dies macht MATLAB zu einer leistungsstarken Plattform für die Entwicklung und

Anwendung von Machine-Learning-Lösungen in verschiedenen Domänen wie Bildverarbeitung, Signalverarbeitung, Finanzen und Biowissenschaften.

2.2 Deep Learning in MATLAB

Deep Learning ist ein Teilbereich des maschinellen Lernens, der auf tiefen neuronalen Netzwerken basiert. Diese Netzwerke bestehen aus mehreren Schichten, die komplexe Muster in umfangreichen Daten erkennen können. Insbesondere bei der Bildverarbeitung zeigt Deep Learning seine Stärken, da es in der Lage ist, automatisch relevante Merkmale aus Bildern zu extrahieren, ohne dass eine manuelle Vorverarbeitung erforderlich ist.

Anwendungen in MATLAB:

In MATLAB stehen zahlreiche Funktionen und Tools zur Verfügung, die die Arbeit mit Deep Learning erleichtern:

- **Convolutional Neural Networks (CNNs):** MATLAB bietet umfassende Unterstützung für die Erstellung und das Training von CNNs. Diese Netzwerke eignen sich hervorragend für Aufgaben wie die Bilderkennung und -klassifikation. Mit der Deep Learning Toolbox können Benutzer Schichten definieren, Netzwerke trainieren und die Modellgüte evaluieren.
- **Transfer Learning:** Um den Aufwand beim Training zu reduzieren, können vortrainierte Modelle wie ResNet, AlexNet oder GoogLeNet genutzt werden. MATLAB ermöglicht das einfache Laden dieser Modelle und ihre Anpassung an neue Datensätze. Transfer Learning ist besonders nützlich, wenn nur wenige Trainingsdaten verfügbar sind.
- **Training und Optimierung:** MATLAB bietet Funktionen, die das Training von Modellen auf leistungsstarken GPUs unterstützen. Dies beschleunigt rechenintensive Prozesse erheblich. Verschiedene Optimierungsverfahren wie Stochastic Gradient Descent (SGD) können konfiguriert werden, um die Modellleistung zu verbessern.
- **Inferenz:** Nach dem Training können die Modelle zur Inferenz verwendet werden, um neue Bilder zu klassifizieren, Objekte zu erkennen oder Bildsegmente zu identifizieren.

Region-based Convolutional Neural Networks (RCNN)

RCNN sind ein leistungsfähiger Ansatz für Aufgaben der Objekterkennung und Bildsegmentierung. Der Kernansatz von RCNN besteht darin, Bildregionen vorzuschlagen und dann ein neuronales Netz auf diese Regionen anzuwenden, um festzustellen, ob sie ein bestimmtes Objekt enthalten. MATLAB bietet umfangreiche Werkzeuge und Funktionen zur Implementierung und Anpassung von RCNN-Modellen.

Grundprinzipien des RCNN-Modells:

RCNN arbeitet in mehreren Schritten, um Objekte in Bildern zu erkennen:

- Zunächst werden mit einem sogenannten Region Proposal Network (RPN) mögliche Objektbereiche identifiziert.
- Diese vorgeschlagenen Regionen werden dann normalisiert und durch ein Convolutional Neural Network (CNN) geleitet.
- Anschließend erfolgt eine Klassifikation der Regionen und eine genaue Positionsbestimmung mittels Bounding Boxes.

RCNN in MATLAB umsetzen:

Die Computer Vision Toolbox von MATLAB bietet Funktionen zur einfachen Erstellung und Anwendung von RCNN-Modellen. Zu den wichtigen Schritten und Möglichkeiten gehören:

- **Datenvorbereitung:** MATLAB unterstützt das Laden und Annotieren von Bilddaten für Objekterkennungsaufgaben. Die Funktion `imageLabeler` vereinfacht die manuelle Annotation und Erstellung von Trainingsdatensätzen.
- **Modelltraining:** Mit der Funktion `trainRCNNObjectDetector` können Benutzer ein RCNN-Modell mit benutzerdefinierten Daten trainieren. Das Training kann auf GPUs ausgeführt werden, um die Performance zu steigern.
- **Transfer Learning:** MATLAB bietet die Möglichkeit, vortrainierte Modelle zu verwenden und auf spezifische Anwendungen zu übertragen. Dies spart Zeit und verbessert die Ergebnisse, wenn nur begrenzte Daten verfügbar sind.

- **Evaluierung:** Funktionen zur Evaluierung des Modells, wie die Berechnung der Mean Average Precision (mAP), helfen dabei, die Qualität des Detektors zu beurteilen.

2.3 Image Processing and Computer Vision in MATLAB

Die Bildverarbeitung und Computer Vision sind essenzielle Technologien, die in Bereichen wie Robotik, Medizin, autonomem Fahren und Sicherheitsüberwachung eine entscheidende Rolle spielen. MATLAB bietet eine leistungsstarke Entwicklungsumgebung mit spezialisierten Toolboxes, die von der Vorverarbeitung bis zur Echtzeitanalyse von Bild- und Videodaten reichen.

Funktionen und spezifische Tools in MATLAB:

- **Bildvorverarbeitung:** Zur Verbesserung der Bildqualität stehen Funktionen wie `imadjust` zur Kontrastanpassung, `histeq` für den Histogrammausgleich und `imfilter` zur Rauschreduktion zur Verfügung. Diese Operationen sind wichtig, um optimale Bedingungen für die weitere Bildanalyse zu schaffen.
- **Merkmalserkennung:** MATLAB bietet leistungsstarke Algorithmen zur Erkennung signifikanter Bildmerkmale. Funktionen wie `edge` für die Kantendetektion und `detectSURFFeatures` zur Erkennung lokaler Merkmale ermöglichen eine detaillierte Analyse von Bildinhalten.
- **Segmentierung:** Die Bildsegmentierung ist ein zentraler Schritt zur Trennung relevanter Objekte vom Hintergrund. MATLAB unterstützt Methoden wie die automatische Schwellwertbildung (`imbinarize`), Clustering-Verfahren (`imsegkmeans`) sowie modellbasierte Ansätze wie aktive Konturen.
- **Objekterkennung und Bildklassifikation:** Neben klassischen Ansätzen wie Template Matching bietet MATLAB Schnittstellen für moderne Deep-Learning-Modelle, darunter YOLO und RCNN, um komplexe Objekterkennungsaufgaben zu lösen.
- **Videoverarbeitung:** Funktionen wie `vision.ForegroundDetector` und `vision.KalmanFilter` erlauben die Echtzeitverarbeitung und Verfolgung bewegter Objekte in Videodaten. Dies ist essenziell für Anwendungen im Bereich der Verkehrsüberwachung oder Videoüberwachung.

- **Integration von Deep Learning:** MATLAB ermöglicht durch die Integration vortrainierter neuronaler Netze eine effiziente Implementierung komplexer Bildverarbeitungsaufgaben. Funktionen zur GPU-Unterstützung beschleunigen den Trainingsprozess erheblich.

MATLAB Image Labeler

Der Image Labeler in MATLAB stellt eine leistungsstarke Anwendung zur Annotation von Bildern dar, die insbesondere in der Bildverarbeitung und im maschinellen Lernen eine signifikante Rolle einnimmt. Die Funktionalität des Programms umfasst die Markierung, Klassifizierung und Labelierung von Objekten in Bildern. Dies ermöglicht die Vorbereitung der Objekte für das Training neuronaler Netze oder andere Analysezwecke. Die Anwendungsgebiete dieser Software sind vielfältig und umfassen unter anderem die automatisierte Objekterkennung, die medizinische Bildanalyse, das autonome Fahren und zahlreiche weitere Bereiche.

Die Anwendung erlaubt verschiedene Arten von Annotationen, darunter Bounding Boxes, Polygonsegmente, pixelweise Maskierungen sowie Szenen-Labels. Bounding Boxes erweisen sich insbesondere bei der Erkennung rechteckiger Objekte als vorteilhaft, wohingegen Polygone eine präzisere Erfassung unregelmäßig geformter Objekte ermöglichen. Die pixelweise Segmentierung ist essenziell für Aufgaben, bei denen jedes einzelne Pixel einem bestimmten Objekt oder Hintergrund zugewiesen werden muss, beispielsweise in der medizinischen Bildverarbeitung.

Die Nutzung des Image Labelers erfolgt intuitiv: Nach dem Start der Applikation können einzelne Bilder oder ganze Bilderserien importiert werden. Im Anschluss werden verschiedene Labels definiert und manuell oder halbautomatisch auf die Bilder angewendet. MATLAB bietet zudem die Möglichkeit, automatische Labeling-Algorithmen zu nutzen, die den Prozess beschleunigen und auf großen Datensätzen effizienter machen. Nach der Annotation können die Daten in verschiedenen Formaten exportiert werden, darunter MATLAB-eigene MAT-Dateien, CSV-Dateien oder Standardformate wie Pascal VOC und COCO, die in vielen Deep-Learning-Frameworks verwendet werden.

Ein wesentlicher Vorteil des Image Labelers liegt in seiner Integration in die MATLAB-Umgebung begründet. Die annotierten Daten können direkt für das Training von neuronalen Netzen wie Faster R-CNN, YOLO oder U-Net genutzt werden. Dies

ermöglicht eine beschleunigte Entwicklung von Modellen für die Objekterkennung oder semantische Segmentierung, ohne dass die Verwendung externer Tools erforderlich ist.

Zusammenfassend bietet der Image Labeler eine effiziente und vielseitige Möglichkeit, Bilddaten für maschinelles Lernen vorzubereiten. Durch seine benutzerfreundliche Oberfläche und die nahtlose Integration in MATLAB ist er ein unverzichtbares Werkzeug für Forscher und Ingenieure, die an Computer-Vision-Projekten arbeiten.

Die Funktion `rectangle` in MATLAB und ihre Anwendung auf Polygone

Die `rectangle`-Funktion in MATLAB dient dazu, rechteckige Formen in einer Grafik oder einem Bild zu zeichnen. Diese Funktion ist besonders nützlich für Anwendungen in der Bildverarbeitung, der computergestützten Analyse und dem maschinellen Lernen, insbesondere wenn es darum geht, Objekte in einem Bild zu markieren. Obwohl sie in erster Linie für rechteckige Markierungen gedacht ist, kann sie in Kombination mit anderen Methoden auch zur Visualisierung und Begrenzung von Polygonen verwendet werden. Da die `rectangle`-Funktion nur rechteckige Markierungen unterstützt, ist sie nicht direkt für die Darstellung von Polygonen geeignet. Dennoch kann sie als Hilfsmittel verwendet werden, um ein umschließendes Rechteck (Bounding Box) um ein Polygon zu zeichnen.

Ein Polygon wird durch eine Menge von Koordinatenpunkten definiert. Um ein Rechteck um dieses Polygon zu ziehen, müssen wir den kleinsten umschließenden Bereich (Bounding Box) berechnen

Ein Polygon wird definiert durch eine Menge von X- und Y-Koordinaten. Die Erstellung des Polygons erfolgt mittels der `Plot`-Funktion, wobei der erste Punkt am Ende wiederholt wird, um das Polygon zu schließen. Die Berechnung der Bounding Box erfolgt durch die Ermittlung der minimalen und maximalen X- und Y-Werte. Daraufhin wird ein Rechteck gezeichnet, welches das Polygon umschließt. Das Ergebnis ist ein blaues Polygon, das von einem roten Rechteck vollständig umschlossen wird.

Zusammenfassung

- Maschinelles Lernen und Deep Learning sind Schlüsseltechnologien für die automatische Analyse von Bilddaten.
- In MATLAB können Sie mit der Deep Learning Toolbox und der Image Processing Toolbox Modelle trainieren und Bilder verarbeiten.
- Image Labeling ist ein wichtiger Schritt, um die Daten für das Training vorzubereiten. Die Image Labeler App in MATLAB erleichtert diesen Prozess.
- Die Kameraerkennung der Pflanze umfasst Bildaufnahme, Vorverarbeitung, Segmentierung und 3D-Rekonstruktion.

Diese Technologien können in deinem Projekt eingesetzt werden, um die dreidimensionale Erfassung von Pflanzenkoordinaten zu automatisieren und die Biomasse sowie die Blattlänge der Pflanzen zu bestimmen.

2.4 Kamerakalibrierung

Die Kalibrierung der Kamera stellt einen essenziellen Schritt in der Bildverarbeitung und der Computer Vision dar, insbesondere in Bezug auf Anwendungen wie 3D-Rekonstruktion, Robotik und visuelles SLAM. Zweck dieser Kalibrierung ist die Korrektur optischer Verzerrungen sowie die Bestimmung der internen und externen Kameraparameter. MATLAB bietet leistungsstarke Werkzeuge zur Kalibrierung, sowohl in Skriptform als auch in Form einer benutzerfreundlichen Applikation.

Kamerakalibrierung mit MATLAB

Die Kalibrierung des Systems erfolgt durch das Erfassen einer Vielzahl von Bildern eines Schachbrettmusters aus unterschiedlichen Blickwinkeln. Mittels der Analyse der Eckpunkte des Schachbretts vermag MATLAB die Brennweite, den Hauptpunkt und die Verzerrungskoeffizienten zu ermitteln. Diese Parameter sind von entscheidender Relevanz, um perspektivische Fehler zu korrigieren und die Genauigkeit der Bildverarbeitung zu erhöhen.

Bedeutung der Kalibrierung

Die exakte Kalibrierung der Kamera ist für eine Vielzahl von Anwendungen von entscheidender Bedeutung, darunter das visuelle SLAM, die 3D-Messtechnik und das maschinelle Sehen. Insbesondere bei Systemen, die auf exakte Tiefeninformationen angewiesen sind, wie beispielsweise RGB-D-Kameras oder Stereo-Kameras, resultiert eine präzise Kalibrierung in einer gesteigerten Genauigkeit und Zuverlässigkeit der Bildverarbeitung.

Zusammenfassung

Die vorliegende Untersuchung befasst sich mit der Kamerakalibrierung unter Verwendung von MATLAB, welche eine präzise Erfassung der Kameraparameter sowie eine Korrektur von Verzerrungen ermöglicht. Die Ermittlung dieser Parameter kann effizient mithilfe der Camera Calibrator App oder durch Skripte erfolgen. Die so ermittelten Parameter bilden die Grundlage für eine präzise Bildverarbeitung und finden in den Bereichen Robotik, Computer Vision und maschinellem Lernen Anwendung.

2.5 Visual SLAM mit RGB-D-Kameras

Grundlagen von Visual SLAM

Grundlagen von Visual *SLAM* (*Simultaneous Localization and Mapping*) ist ein Verfahren, das die simultane Lokalisierung einer Kamera und die Kartierung der Umgebung in Echtzeit ermöglicht. Die Erfassung von Farbinformationen (RGB) und Tiefendaten (D) mittels RGB-D-Kameras, wie beispielsweise Microsoft Kinect oder Intel RealSense, ermöglicht eine präzisere 3D-Rekonstruktion.

Methoden des Visual SLAM

Moderne SLAM-Methoden lassen sich in feature-basierte Verfahren und direkte Methoden unterteilen.

- Feature-basierte SLAM-Ansätze, wie sie etwa durch ORB-SLAM repräsentiert werden, extrahieren markante Punkte aus Bildern und gleichen diese zwischen mehreren Aufnahmen ab.

- Direkte Verfahren, wie sie LSD-SLAM anwendet, analysieren Helligkeitsveränderungen und eignen sich insbesondere für detailreiche Umgebungen.
- RGB-D-SLAM-Methoden, wie sie etwa ElasticFusion oder KinectFusion anwenden, kombinieren Farbinformationen und Tiefendaten, um eine höhere Präzision bei der Kartenerstellung zu erreichen.

Herausforderungen und Entwicklungen

Trotz des technologischen Fortschritts bestehen nach wie vor Herausforderungen, darunter hohe Rechenanforderungen, Schwierigkeiten bei spiegelnden oder transparenten Oberflächen sowie dynamische Szenen, die zu Fehlern in der Kartierung führen können. Fortschritte in den Bereichen KI und Deep Learning tragen jedoch zu einer verbesserten Robustheit von SLAM-Systemen bei und eröffnen neue Anwendungsmöglichkeiten in den Bereichen Robotik, Augmented Reality und autonomer Navigation.

3. Hauptteil

3.1 Aufbau der Prüfstand

Aufbau der Kamera



Abbildung 1: Aufbau der Kamera

Fixierung der Kamera

Diesem Projekt hat Sensorbewegungsplattform mit 5 Freiheitsgraden (DOF) Eine Kamera mit Schwenk- und Neigefreiheitsgraden. Diese Plattform sollte in der Lage sein, bestehende Forschungsausrüstung aus allen möglichen Positionen mit Hilfe einer Kamera zu beobachten.

Die Kamera wird sicher an der vorgesehenen Halterung befestigt, um eine stabile und präzise Aufnahme der Bilder zu gewährleisten. Dabei wird darauf geachtet, dass die Ausrichtung exakt justiert ist, um Verzerrungen oder unerwünschte Bewegungen zu vermeiden.

Kabelmanagement

Das Kabelmanagement hat das Ziel, alle Kabel ordentlich und organisiert zu halten sowie sie vor Beschädigungen zu schützen, wodurch ihre Lebensdauer verlängert wird. In diesem Projekt umfassen die Kabel die Datenübertragungskabel für Motorsteuerungen, Sensoren und Kamera.

Da die Motoren auf den Linearachsen der X-, Y- und Z-Achse montiert sind, muss die Länge der mit den Motoren verbundenen Kabel die maximale Bewegungsreichweite der Lineareinheit erreichen, um eine sichere Verbindung zu gewährleisten.

Für das Kabel-Management werden alle Kabel ordentlich verlegt und fixiert, sodass sie nicht lose herumhängen oder die Beweglichkeit des Systems beeinträchtigen. Dies dient nicht nur der Sicherheit, sondern auch der Vermeidung möglicher Signalstörungen oder physischer Schäden an den Verbindungen.

Neue Profile an dem Prüfstand

Die Hauptstruktur wurde so entworfen, dass sie aus Aluminiumprofilen konstruiert wird.

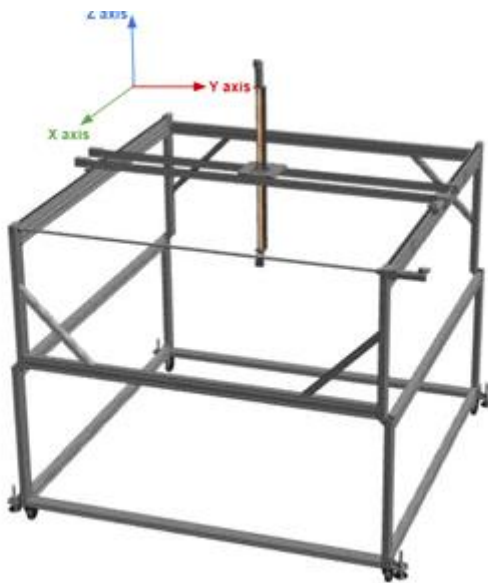


Abbildung 2: Aufbau des Prüfstands

Erweiterung des Prüfstands zur Optimierung der Bewegung des Zahnriemens

Um die Bewegung des Zahnriemens (Timing Belt) zu erleichtern, wurden neue Aluminiumprofile am Prüfstand befestigt, wie in den Bildern dargestellt. Diese zusätzlichen Profile dienen der Stabilisierung der Struktur und gewährleisten eine reibungslose Führung des Zahnriemens entlang der vorgesehenen Bewegungsachsen.

Durch diese Anpassung wird die Effizienz des Systems verbessert, da unerwünschte Vibrationen oder Widerstände reduziert werden. Zudem trägt die neue Konstruktion dazu bei, die Langlebigkeit des Zahnriemens und der gesamten Mechanik zu erhöhen.



Abbildung 3: Erweiterung des Prüfstands

Band von Motor Z aufgelöst und neu befestigt:

Nach der Befestigung neuer Profile an dem Prüfstand wurde die Neujustierung des Zahnriemens am Z-Motor durchgeführt. Der Zahnriemen des Z-Motors wurde gelöst und anschließend neu befestigt, um eine optimale Spannung und Funktionalität sicherzustellen. Diese Anpassung war notwendig, um eine präzise Bewegung entlang der Z-Achse zu gewährleisten und möglichen Verschleiß oder eine Fehlfunktion zu vermeiden. Durch die erneute Befestigung des Zahnriemens wird die Leistung des Antriebs verbessert, wodurch die Stabilität und Genauigkeit des gesamten Systems erhöht werden.

Befestigung von Magnet und Magnetsensor für die Tilt-Bewegung

Ein Magnet und ein Magnetsensor wurden zur Erfassung der Tilt-Bewegung befestigt. Diese Komponenten ermöglichen eine präzise Erkennung der Neigungswinkel und tragen zur genauen Steuerung der Bewegung bei.

Die Platzierung des Magnetsensors wurde so gewählt, dass eine zuverlässige und wiederholbare Messung gewährleistet ist. Dadurch wird die Systemstabilität verbessert und eine präzisere Positionierung der Kamera ermöglicht.

3.2 Fehlerdiagnose und Lösungsansätze

Zu Beginn des Projekts erhielten wir von unserem Betreuer einen bestehenden Arduino- und Processing-Code aus einer früheren Masterarbeit, der die Grundlage für unsere Arbeiten bildete. Um mit der Umsetzung fortzufahren, luden wir die notwendigen IDEs herunter, darunter die Arduino IDE für die Programmierung des Mikrokontrollers und die Processing IDE für die Steuerung der Benutzeroberfläche. Zudem importierten wir die benötigten Bibliotheken, wie zum Beispiel *AccelStepper.h* für die Motorsteuerung in der Arduino IDE und *processing.serial*., *controlP5*. sowie *java.awt.Font* in Processing.

Leider stellte sich heraus, dass der Code nicht wie erwartet funktionierte. Unser erster Schritt war es, die Ursache des Problems zu identifizieren. Ein erster Verdacht galt den Pin-Definitionen im Code. Es konnte sein, dass die Motorsteuerung in den Codezeilen auf Pins 1 und 2 gesetzt war, obwohl die Motoren physikalisch an den Pins 10 und 11 des Arduino-Boards angeschlossen waren. Nach Anpassung dieser Pin-Definitionen zeigte sich jedoch weiterhin keine Funktionalität, was uns dazu veranlasste, weiter nach der Fehlerquelle zu suchen.

Im nächsten Schritt überprüften wir die Hardware-Komponenten, um mögliche Defekte auszuschließen. Dabei fiel auf, dass einige der Magnet-Sensoren inkorrekte Werte lieferten. Durch Beobachtung der vom Sensor ausgegebenen Werte im Serial Monitor der Arduino IDE stellten wir fest, dass die Werte zwischen 1 und 0 schwankten, ohne dass der Sensor betätigt wurde. Dies deutete auf einen Fehler im Sensor hin. Eine weitere Analyse zeigte, dass der Magnet-Sensor durch das Magnetfeld des Motors beeinflusst wurde, was zu ungenauen und teils fehlerhaften Messwerten führte. Aufgrund dieses Problems entschlossen wir uns, die Magnet-Sensoren durch mechanische Limit-Switches zu ersetzen, die zuverlässiger in der Funktion und weniger anfällig für Störungen durch Magnetfelder waren.

Dieser Schritt war entscheidend, um die Funktionsfähigkeit des Systems sicherzustellen und die Hardware auf stabile und fehlerfreie Komponenten umzustellen. In den folgenden Abschnitten werden die weiteren Schritte zur Optimierung des Systems und zur Integration der neuen Hardware beschrieben.

3.2 Entwicklung und Integration eines Gehäuses für die Micro-Limit-Switches

Nachdem wir die Micro-Limit-Switches erfolgreich erstellt hatten, bestand der nächste Schritt darin, ein passendes Gehäuse für diese zu entwerfen. Dafür haben wir mehrere Varianten skizziert und schließlich eine optimale Lösung ausgewählt. Die technische Zeichnung des Gehäuses wurde mit der Software *SIEMENS NX* angefertigt, um eine präzise und professionelle Konstruktion zu gewährleisten.

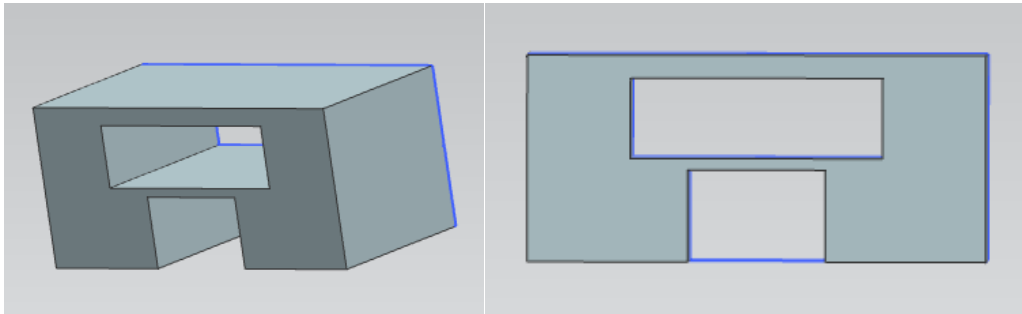


Abbildung 4: 3D-Modell des Gehäuses

Im Anschluss haben wir das Gehäuse mittels *3D-Druck* am Lehrstuhl gefertigt. Nach der Fertigung wurden die vorhandenen Magnetsensoren entfernt und durch neue Sensoren ersetzt, welche anschließend in die Leitungssätze eingelötet wurden. Danach erfolgte die Montage der Sensoren in den gedruckten Gehäusen, welche wiederum am Prüfstand befestigt wurden.



Abbildung 5: Der Limit-Switch im Gehäuse am Prüfstand

Nach der mechanischen Integration testeten wir den Code erneut, jedoch ohne Erfolg – die Software funktionierte nicht wie erwartet. Die genaue Ursache des Problems konnte nicht eindeutig identifiziert werden. Unsere Vermutung war, dass bestimmte Funktionen des Codes auf Werte von Ultraschallsensoren zugriffen, die

möglicherweise in einem vorherigen Projekt verwendet wurden, für unser aktuelles Projekt jedoch nicht relevant waren. Aufgrund dieser Unsicherheiten haben wir beschlossen, einen neuen *Arduino-Code* zu schreiben und darauf basierend einen passenden *Processing-Code* zu entwickeln, um eine fehlerfreie Funktionalität sicherzustellen.

3.3 Beschreibung der Arduino-Code-Entwicklung

In diesem Abschnitt wird der entwickelte Arduino-Code detailliert beschrieben, der die Steuerung von fünf Schrittmotoren in einem robotischen oder CNC-System ermöglicht. Der Fokus liegt auf einer sicheren und präzisen Steuerung der Bewegungen, unterstützt durch Limit-Schalter, softwareseitige Begrenzungen und eine benutzerfreundliche serielle Schnittstelle. Die wichtigsten Aspekte des Codes werden im Folgenden schrittweise erläutert.

Der Code wurde entwickelt, um fünf Schrittmotoren für die Steuerung eines mehrdimensionalen Bewegungssystems zu kontrollieren. Jede der fünf Achsen (X, Y, Z, Neigung und Drehung) erfüllt spezifische Aufgaben:

- X-, Y- und Z-Achsen: Steuerung der Bewegungen in der horizontalen, vertikalen und Tiefenebene.
- Neigungsachse: Kippen des Systems für variable Neigungen.
- Drehachse: Rotation des Systems um eine feste Achse.

Die Motorsteuerung erfolgt durch die AccelStepper-Bibliothek, die eine präzise Kontrolle von Geschwindigkeit, Beschleunigung und Richtung ermöglicht. Über die serielle Schnittstelle können Bewegungsbefehle in Echtzeit eingegeben und verarbeitet werden.

Um Schäden an der Hardware zu verhindern, wurden sowohl physische als auch softwareseitige Sicherheitsmaßnahmen integriert:

- Limit-Schalter: Jeder Schrittmotor ist mit einem Limit-Schalter ausgestattet, der das Überschreiten der mechanischen Grenzen verhindert. Sobald ein Limit-Schalter ausgelöst wird, stoppt der entsprechende Motor, und seine Position wird auf null gesetzt.

- Bewegungsgrenzen: Zusätzlich zu den Limit-Schaltern gibt es softwareseitige Begrenzungen, die die maximal zulässigen Schritte in negativer Richtung definieren. Diese Grenzen sind individuell an die Achsen angepasst und stellen sicher, dass die Motoren nicht überlastet werden.

In der `setup()`-Funktion wird das System initialisiert. Dazu gehören:

- Definition der maximalen Geschwindigkeit und Beschleunigung für jeden Motor.
- Konfiguration der Limit-Schalter als Eingänge mit Pull-Up-Widerständen.
- Ausführung der Funktion `initializeMotors()`, um die Motoren an ihre Ausgangspositionen zu fahren. Diese Funktion bewegt die Motoren vorwärts, bis die jeweiligen Limit-Schalter ausgelöst werden. Sobald ein Schalter aktiviert wird, stoppt der Motor und setzt seine Position auf null. Diese Initialisierung stellt sicher, dass das System in einem definierten Zustand startet.

Die Bewegung der Motoren wird durch Benutzereingaben über die serielle Schnittstelle gesteuert. Die Eingaben bestehen aus der Motornummer und der Anzahl der Schritte. Nachfolgend sind die wesentlichen Funktionen zur Steuerung beschrieben:

- `moveMotor()`: Diese Funktion führt die Bewegung eines Motors aus und überprüft dabei, ob die Bewegungsgrenzen eingehalten werden. Überschreitungen werden blockiert, und eine Fehlermeldung wird ausgegeben.
- `movePanMotor()`: Diese Funktion wurde speziell für die Drehachse entwickelt. Sie stellt sicher, dass die Position des Motors innerhalb eines vordefinierten Bereichs bleibt.

Die serielle Schnittstelle ermöglicht eine einfache und direkte Steuerung der Bewegungen in folgendem Format: `<Motornummer> <Schritte>` (z. B. `1 100` bewegt den ersten Motor um 100 Schritte vorwärts).

Der Code bietet eine benutzerfreundliche Möglichkeit zur Steuerung des Systems. Eingaben werden überprüft, und fehlerhafte oder ungültige Befehle werden ignoriert. In solchen Fällen wird dem Benutzer eine Fehlermeldung angezeigt. Diese Maßnahmen verbessern die Zuverlässigkeit des Systems und minimieren das Risiko von Fehlbedienungen.

Der entwickelte Arduino-Code bietet eine zuverlässige und flexible Lösung für die Steuerung eines robotischen oder CNC-Systems mit fünf Achsen. Durch die Kombination aus präziser Bewegungssteuerung, umfassenden Sicherheitsmechanismen und einer benutzerfreundlichen seriellen Schnittstelle wird eine sichere und effiziente Nutzung ermöglicht. Das System ist sowohl für experimentelle Anwendungen als auch für produktive Umgebungen geeignet und stellt eine solide Grundlage für weitere Entwicklungen dar.

Beschreibung der Processing-Code-Entwicklung

Dieser Abschnitt beschreibt die Entwicklung eines Processing-Codes, der speziell dazu dient, Benutzereingaben in Form von Bewegungs- und Rotationsdaten für ein Arduino-gesteuertes Gerät zu verarbeiten. Das Ziel dieses Codes ist es, einfache, benutzerfreundliche Eingaben in Zentimetern und Grad entgegenzunehmen, diese in ein für den Arduino verständliches Format umzuwandeln und sie schließlich über eine serielle Schnittstelle zu senden. Nachfolgend wird die Funktionsweise des Codes im Detail erläutert.

Der Code ist in der Programmiersprache Processing geschrieben, die oft für grafische und serielle Kommunikationsanwendungen verwendet wird. Hier wird eine grafische Benutzeroberfläche bereitgestellt, über die der Benutzer Bewegungsparameter eingeben kann. Diese Parameter umfassen:

- Strecke: Die Entfernung, die das Gerät in Zentimetern zurücklegen soll.
- Drehwinkel: Der Winkel, um den sich das Gerät drehen soll, angegeben in Grad.

Die vom Benutzer eingegebenen Werte sind intuitiv gestaltet und werden in Zentimetern (für die Bewegung) und Grad (für die Drehung) eingegeben. Diese Maßeinheiten sind jedoch nicht direkt für das Gerät geeignet, da es in Schritten (Steps) operiert. Schritte sind die kleinsten Bewegungseinheiten des Geräts, oft definiert durch den Steppermotor.

- Berechnung der Schritte aus Zentimetern: Der Code verwendet eine vorher definierte Konversionsformel oder einen Faktor, um Zentimeter in Schritte umzuwandeln. Dieser Faktor hängt von der physikalischen Konstruktion des Geräts ab, z. B. der Schrittlänge des Motors und der Mechanik.
- Berechnung der Schritte aus Grad: Ähnlich wird der Drehwinkel von Grad in entsprechende Schritte umgerechnet, basierend auf der Kalibrierung des Drehmechanismus.

Sobald die Werte umgewandelt wurden, erstellt der Code ein Datenpaket, das für die serielle Übertragung geeignet ist. Diese Kommunikation läuft über die serielle Schnittstelle, wobei die folgenden Aspekte berücksichtigt werden:

- Datensatz-Format: Die umgewandelten Schritte werden in einer bestimmten Reihenfolge in einem String formatiert, beispielsweise als "X-Schritte, Y-Schritte".
- Senden der Daten: Über die serielle Schnittstelle werden die Datenpakete an das angeschlossene Arduino-Gerät gesendet. Das Arduino empfängt die Daten, interpretiert sie und führt die entsprechenden Bewegungen aus.

Die grafische Benutzeroberfläche bietet eine einfache Methode, um Werte einzugeben und die Kommunikation zu starten. Typische Elemente dieser Benutzeroberfläche fassen Textfelder für die Eingabe, Schaltflächen zum Starten der Übertragung und Statusanzeigen für die Verbindung um.

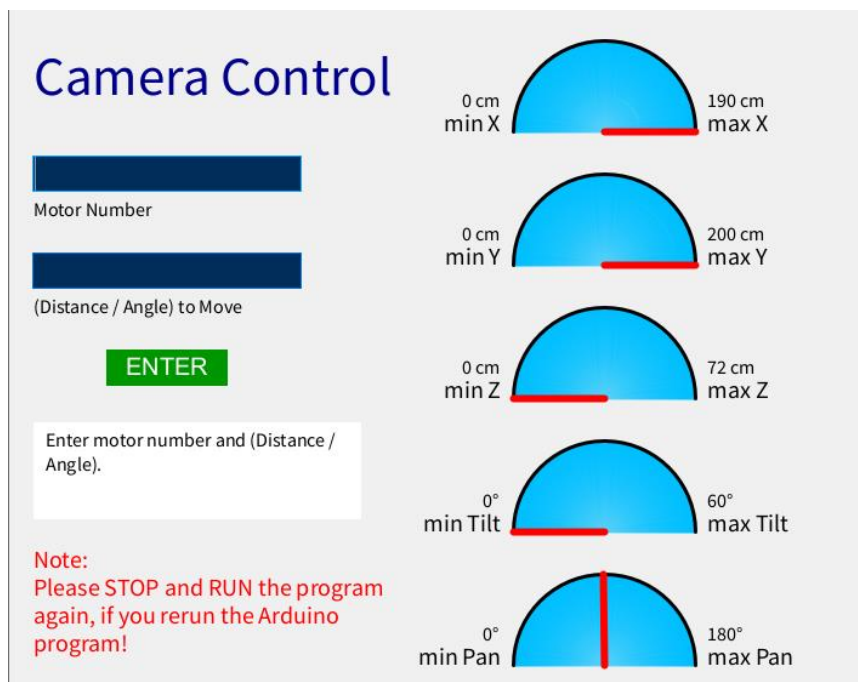


Abbildung 6: Graphical User Interface zur Steuerung der Kameraposition

Dieser Processing-Code zeigt, wie komplexe Benutzeranforderungen in eine maschinenlesbare Form übersetzt werden können. Durch die Umrechnung von Zentimetern und Grad in Schritte und die anschließende Übertragung über die serielle Schnittstelle entsteht eine nahtlose Verbindung zwischen Benutzer und Hardware, die präzise Steuerungen ermöglicht.

3.4 Bildverarbeitung

Das Ziel dieses Projekts war die Entwicklung eines dreidimensionalen Messsystems zur präzisen Erfassung von Pflanzenkoordinaten. Durch den Einsatz moderner Sensortechnologien sollten Pflanzenstrukturen exakt vermessen und digitalisiert werden. Dies ermöglicht eine verbesserte Analyse des Pflanzenwachstums, optimierte Anbauprozesse und eine nachhaltigere Ressourcennutzung.

Image Labeler

Zunächst werden mehrere Bilder für das Objekt aufgenommen, um das Machine Learning zu ermöglichen.

Der Image Labeler, dessen Funktionsweise im Kapitel zum Stand der Technik beschrieben wird, wurde zur Annotation mehrerer Bilder verwendet (siehe Abbildung 4). Im Rahmen des Trainings wurden alle Bilder in Matlab mit Rechtecken und Polygonen versehen.

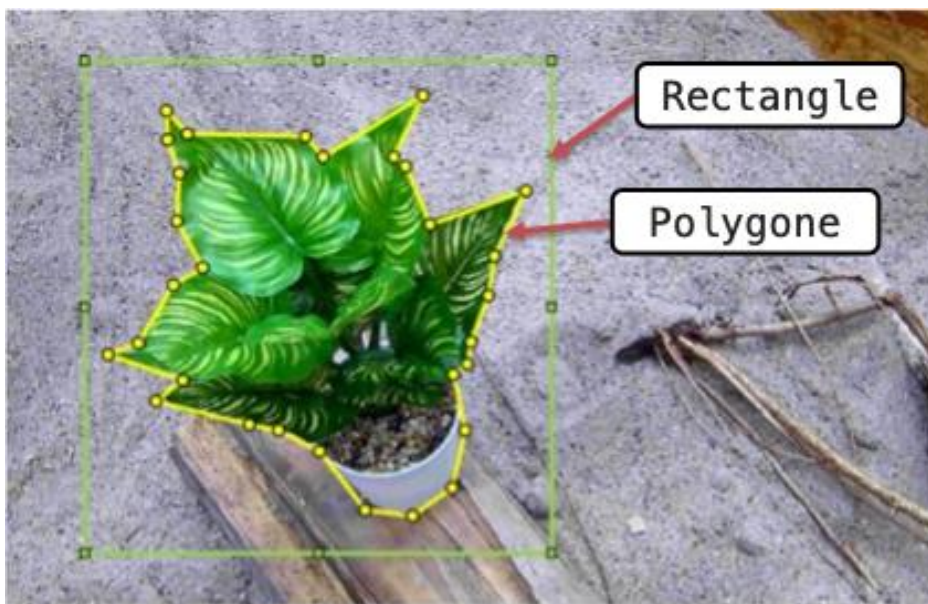


Abbildung 7: Funktionsweise des Image Labelers

Datenvorbereitung

Nach der Annotation der Bilder erfolgt der Export der erfassten Daten als Tabelle. Diese enthält wesentliche Informationen wie die Bounding-Box-Koordinaten der Objekte, die zugehörigen Bilddateinamen, Objektbezeichnungen (Labels) sowie Masken für die pixelgenaue Segmentierung. Der Export erfolgt in einem strukturierten Format, das sich für die Weiterverarbeitung eignet.

Zur weiteren Verarbeitung wird ein MATLAB-Skript verwendet, das im Anhang dieser Arbeit unter der Bezeichnung Boxes, Name-File, Labels, Masks aufgeführt ist.

Dieses Skript übernimmt mehrere Aufgaben:

- Extraktion und Umwandlung der Annotationen: Die Bounding-Box-Koordinaten (x, y, Breite, Höhe) werden aus der Datei ausgelesen und strukturiert abgelegt.
- Verknüpfung mit den entsprechenden Bilddateien: Die Annotationen werden den jeweiligen Bildern zugeordnet, indem Dateinamen korrekt verknüpft werden.
- Die Kategorisierung der Objekte erfolgt durch das Auslesen der in der Annotation enthaltenen Labels und die Nutzbarmachung für weitere Berechnungen oder Modelltrainings.
- Falls polygonale Annotationen vorliegen, werden Masken erzeugt, um eine pixelgenaue Objekterkennung zu ermöglichen.

Das Skript dient der strukturierten Vorbereitung der Daten für den Einsatz in maschinellen Lernmodellen, insbesondere für Objekterkennungsalgorithmen wie YOLO oder Mask R-CNN. Durch die automatisierte Verarbeitung wird sichergestellt, dass die Annotationen in einem standardisierten Format vorliegen und direkt für das Training oder die Analyse verwendet werden können.

Trainingsphase mit R-CNN

Nach der Datenaufbereitung folgt die Trainingsphase, in der die annotierten Bilddaten zur Objekterkennung mit Hilfe eines *Region-based Convolutional Neural Network* (R-CNN) verwendet werden. R-CNN gehört zu den modernen Methoden der Bildverarbeitung und entspricht dem aktuellen Stand der Technik. Das Modell verwendet Region Proposal Networks (RPNs), um relevante Bildbereiche zu identifizieren und anschließend mit einem neuronalen Netz zu klassifizieren.

Das Training erfolgt in MATLAB unter Verwendung der zuvor erstellten Bounding-Box-Daten und Masken. Dabei werden die folgenden Schritte durchlaufen:

- Datenladen und Vorverarbeitung: Die annotierten Bilder und die zugehörigen *Bounding Boxes* bzw. Segmentierungsmasken werden in das Modell eingespeist.

- Merkmalsextraktion mit *Convolutional Neural Networks* (CNNs): Das Modell extrahiert relevante Merkmale aus den Bildern, um Objekte zu erkennen.
- *Region Proposal Network* (RPN): Potenzielle Objektregionen werden automatisch generiert.
- Klassifikation und Bounding-Box-Regression: Das Modell klassifiziert die Objekte und verfeinert ihre Position.
- Modelltraining und Optimierung: Das Netz wird iterativ trainiert, um die Erkennungsgenauigkeit zu verbessern.

Das zugehörige MATLAB-Skript, das den Trainingsprozess implementiert, liegt dieser Arbeit bei. Es enthält alle notwendigen Schritte, um das Training durchzuführen und die Leistung des Modells zu evaluieren.

Kamerakalibrierung

Die *Camera Calibrator* App in MATLAB offeriert eine benutzerfreundliche Option zur Kamerakalibrierung. Über eine grafische Benutzeroberfläche können importierte Schachbrettbilder erfasst und die Eckpunkte automatisch oder manuell erkannt werden. Im Anschluss werden durch die Applikation essenzielle Kameraparameter wie die Brennweite, Verzerrungskoeffizienten und die Projektionsmatrix ermittelt. Die Resultate können gespeichert und für weitere Bildverarbeitungsaufgaben verwendet werden.

Die nachfolgenden Abbildungen (Abbildung 8) veranschaulichen den typischen Ablauf der Kalibrierung:

- **Import der Schachbrettbilder:** Aufnahmen des Schachbrettmusters werden in die App geladen.
- **Automatische oder manuelle Erkennung der Schachbrettecken:** Die App erkennt die Ecken des Musters, um Verzerrungen zu berechnen.
- **Berechnung der Kameraparameter:** Wichtige Parameter wie Brennweite, Verzerrungswerte und die Projektionsmatrix werden ermittelt.
- **Visualisierung der Kalibrierungsergebnisse:** Die berechneten Parameter können überprüft und für weitere Anwendungen gespeichert werden.

Diese Methode vereinfacht die Kalibrierung und ermöglicht eine präzise Korrektur von Bildverzerrungen.

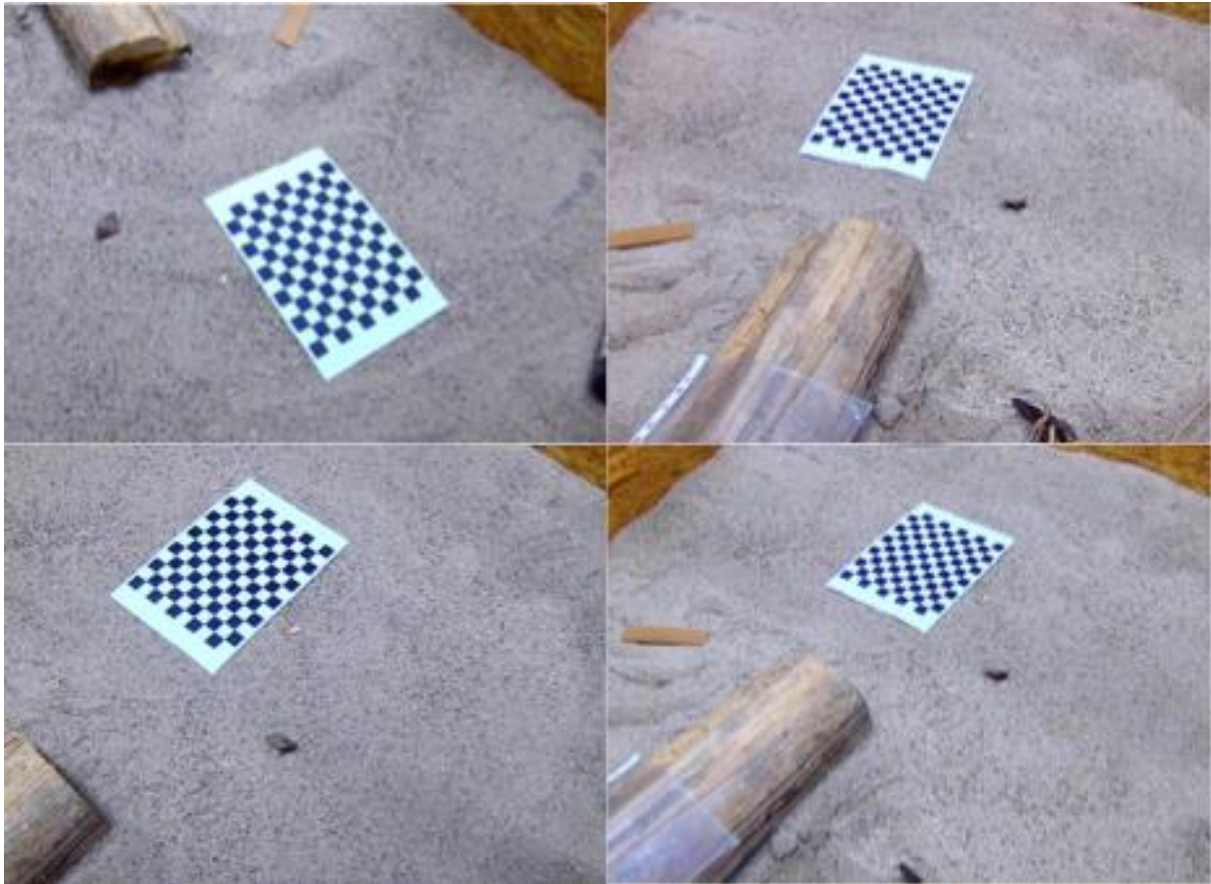


Abbildung 8: Bilder für die Kamerakalibrierung mit der Nutzung von Schachbrett

Visual SLAM und 3D Vision

Zunächst erfolgt die Kombination des Ergebnisses der Kamerakalibrierung mit dem Ergebnis des Trainings in einem Skript, welches im Anhang dieser Arbeit zu finden ist und eine präzise Rekonstruktion der Umgebung ermöglicht. Die Kalibrierungsparameter, wie die intrinsischen und extrinsischen Kamerawerte, werden dabei verwendet, um Bildverzerrungen zu korrigieren und eine zuverlässige Tiefenschätzung zu gewährleisten.

Im darauffolgenden Schritt wird die Merkmalsextraktion und -verfolgung durchgeführt. Algorithmen wie ORB (*Oriented FAST and Rotated BRIEF*) oder SIFT (*Scale-Invariant Feature Transform*) identifizieren markante Punkte in den aufgenommenen Bildern (Abbildung 9) und verfolgen deren Bewegung über mehrere Frames hinweg.



Abbildung 9: Zwei Bilder der Pflanze mit zwei verschiedene Position

In einem anschließenden Optimierungsprozess werden diese Informationen verarbeitet, um eine konsistente Trajektorie der Kamera zu berechnen.

Parallel dazu wird eine dichte oder spärliche 3D-Punktwolke (Abbildung 10) generiert, die die räumliche Struktur der Umgebung beschreibt. Hierbei werden Verfahren wie *Structure from Motion* (SfM) oder *Multi-View Stereo* (MVS) angewendet, um die Tiefeninformationen aus mehreren Blickwinkeln zu berechnen.

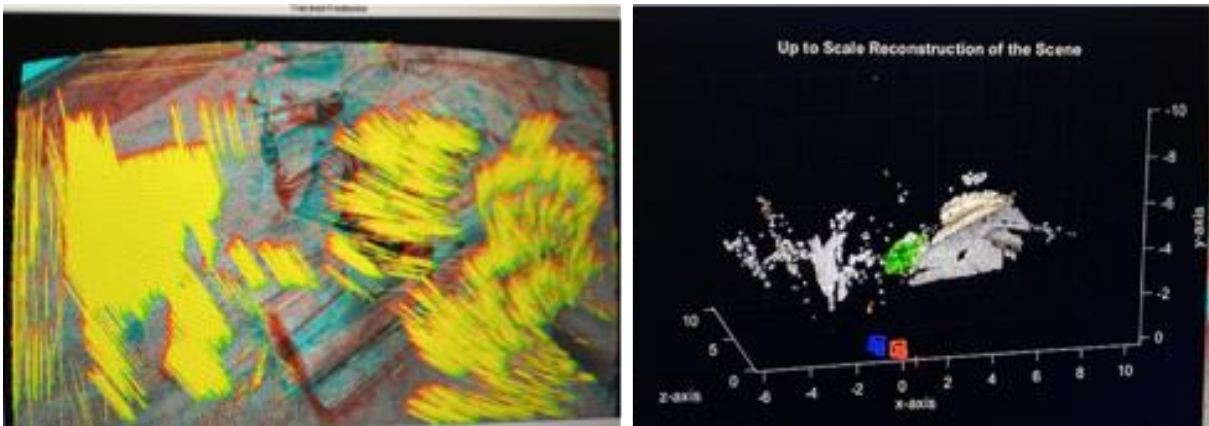


Abbildung 10: Merkmalsextraktion und -verfolgung (Links), 3D-Punktwolke der Szene (Rechts)

Schließlich werden die ermittelten Kamerapositionen und 3D-Punkte in ein *Simultaneous Localization and Mapping* (SLAM)-System integriert. Dieses ermöglicht die kontinuierliche Positionsbestimmung der Kamera in einer unbekannten Umgebung sowie die simultane Erstellung einer 3D-Karte. Die nachfolgenden Abbildungen veranschaulichen beispielhaft die Verarbeitungsschritte von der Bildaufnahme bis zur 3D-Rekonstruktion.

4. Ergebnisse und Auswertung

Im Rahmen dieser Arbeit wurde ein Deep-Learning-Modell zur dreidimensionalen Erfassung von Pflanzenkoordinaten entwickelt und trainiert. Ziel war es, mit Hilfe von Region-based Convolutional Neural Networks (R-CNN) eine präzise Objekterkennung durchzuführen. Nach erfolgreicher Datenannotation und -aufbereitung wurde das Modell trainiert und getestet.

Die Trainingsdaten werden in einer Datei mit dem Namen ‚net‘ gespeichert, die alle Daten zu unserem Objekt enthält. Das Mask R-CNN Modell wurde mit dem Befehl *trainMaskRCNN* in MATLAB trainiert. Dabei wurde das Backbone-Netz eingefroren, um eine stabile Merkmalsextraktion zu gewährleisten. Das Modell durchlief mehrere Trainingszyklen, wobei die Gewichte der Masken- und Objekterkennungsschichten optimiert wurden.

Nach dem Training wurde das Modell an neuen Bildern getestet. Dazu wurde ein Beispielbild in MATLAB geladen und für die Objekterkennung vorbereitet. Das trainierte Modell wurde auf ein Testbild angewendet, um Objekte und deren Positionen zu identifizieren. Für jedes erkannte Objekt wurde eine Maske, ein Label und ein Wahrscheinlichkeitswert berechnet. Die Ergebnisse zeigen, dass das Modell in der Lage ist, Objekte präzise zu segmentieren und zu bestimmen.

Die erkannten Objekte wurden mit farbigen Masken (Abbildung 11) über das Bild gelegt, um die Ergebnisse visuell darzustellen. Dies ermöglicht eine einfache Überprüfung der Genauigkeit der Segmentierung. Das Modell war in der Lage, die gewünschten Objekte zuverlässig zu erkennen und ihre Grenzen genau zu bestimmen.



Abbildung 11: Das Ergebnis für die Erkennung der Pflanze

Der Pfeil in der Abbildung markiert eine der erkannten Masken, welche die exakte Position und Form des identifizierten Objekts darstellt. Diese visuelle Darstellung ermöglicht eine einfache Überprüfung der Segmentierungsergebnisse. Die hohe Genauigkeit der Maskierung demonstriert, dass das Modell erfolgreich darin ausgebildet wurde, Objekte zu lokalisieren und von ihrer Umgebung zu differenzieren.

Die visuelle Darstellung veranschaulicht die Leistungsfähigkeit des Modells hinsichtlich der Erkennung und Segmentierung von Objekten. Der markierte Bereich demonstriert eine besonders präzise Struktur, während weitere Analysen erforderlich sind, um kleinere Ungenauigkeiten zu optimieren.

Visual SLAM mit RGB-D-Kameras

Die Resultate demonstrieren eine effektive Implementierung der grundlegenden Schritte im Visual SLAM. Die Merkmalsextraktion wurde präzise durchgeführt, und die Bewegungsverfolgung der Merkmale über mehrere Frames ist konsistent. Die berechneten Kamerapositionen sind stabil, und die erzeugte 3D-Punktwolke bietet eine skalierte Darstellung der Umgebung.

Die Genauigkeit der Rekonstruktion ist als solide zu bewerten, und die Daten liefern wertvolle Informationen für die weitere Verarbeitung. Potenzielle Verbesserungen ergeben sich in der Reduktion von Rauschelementen sowie in der Verfeinerung der Punktwolke für detailliertere Anwendungen. Die Ergebnisse spiegeln insgesamt eine robuste Grundlage für Visual SLAM und 3D-Vision wider und können als Ausgangspunkt für weiterführende Optimierungen genutzt werden.

Die vorliegende Untersuchung demonstriert die erfolgreiche Implementierung der grundlegenden Schritte eines Visual SLAM-Systems, einschließlich der Merkmalsextraktion und der 3D-Rekonstruktion. Für weiterführende Anwendungen, wie beispielsweise Roboternavigation oder Umgebungsmodellierung, könnte die Punktwolke verfeinert und die Genauigkeit der Merkmalsverfolgung weiter verbessert werden.

5. Anhang

Arduino-Code

```
#include <AccelStepper.h>
```

```
// Define stepper motors and drivers
```

```
AccelStepper stepper1(1, 9, 8); // Stepper X
```

```
AccelStepper stepper2(1, 7, 6); // Stepper Y
```

```
AccelStepper stepper3(1, 5, 4); // Stepper Z
```

```
AccelStepper stepper4(1, 13, 12); // Stepper Tilt
```

```
AccelStepper stepper5(1, 11, 10); // Stepper Pan
```

```
// Define limit switches
```

```
#define limitSwitchX1 53 // Stepper X1
```

```
#define limitSwitchY1 35 // Stepper Y1
```

```
#define limitSwitchZ1 39 // Stepper Z1
```

```
#define limitSwitchTilt 47 // Stepper Tilt
```

```
#define limitSwitchPan1 43 // Stepper Pan
```

```
// Variables to track negative steps for each motor
```

```
int negativeStepsX = 0;
```

```
int negativeStepsY = 0;
```

```
int negativeStepsZ = 0;
```

```
int negativeStepsTilt = 0;
```

```
const int MAX_NEGATIVE_STEPS = 25000; // Maximum allowed negative steps  
for motor 1
```

```
const int MAX_NEGATIVE_STEPS_MOTOR2 = 5300; // Maximum allowed negative  
steps for motor 2
```

```
const int MAX_NEGATIVE_STEPS_MOTOR3 = 2000; // Maximum allowed negative  
steps for motor 3
```

```
const int MAX_NEGATIVE_STEPS_MOTOR4 = 800; // Maximum allowed negative
steps for motor 4
```

```
const int MAX_STEPS_PAN = 800;           // Maximum steps for motor 5 (Pan)
```

```
void setup() {
```

```
  // Initialize Serial Monitor
```

```
  Serial.begin(9600);
```

```
  Serial.println("Initializing motors...");
```

```
  // Set maximum speed and acceleration for all steppers
```

```
  stepper1.setMaxSpeed(200);
```

```
  stepper1.setAcceleration(2000);
```

```
  stepper2.setMaxSpeed(40);
```

```
  stepper2.setAcceleration(2000);
```

```
  stepper3.setMaxSpeed(40);
```

```
  stepper3.setAcceleration(2000);
```

```
  stepper4.setMaxSpeed(100);
```

```
  stepper4.setAcceleration(500);
```

```
  stepper5.setMaxSpeed(50);
```

```
  stepper5.setAcceleration(500);
```

```
  // Invert step and direction pins for all steppers
```

```
  stepper1.setPinsInverted(true, true, false);
```

```
  stepper2.setPinsInverted(true, true, false);
```

```
  stepper3.setPinsInverted(true, true, false);
```

```
  stepper4.setPinsInverted(true, true, false);
```

```

stepper5.setPinsInverted(true, true, false);

// Configure limit switch pins as inputs with pull-up resistors
pinMode(limitSwitchX1, INPUT_PULLUP);
pinMode(limitSwitchY1, INPUT_PULLUP);
pinMode(limitSwitchZ1, INPUT_PULLUP);
pinMode(limitSwitchTilt, INPUT_PULLUP);
pinMode(limitSwitchPan1, INPUT_PULLUP);

// Initialize motors' positions
initializeMotors();

Serial.println("Initialization complete.");

Serial.println("Enter the motor number (1, 2, 3, 4, or 5) and steps to move (negative
for reverse):");
}

void loop() {
    // Check if data is available in the Serial Monitor
    if (Serial.available() > 0) {
        // Read input
        String input = Serial.readStringUntil('\n');
        input.trim(); // Remove extra spaces or newlines

        // Split the input into motor number and steps
        int spaceIndex = input.indexOf(' '); // Find the space separator
        if (spaceIndex == -1) {
            Serial.println("Invalid input format. Use: <motor number> <steps>");
            return;
        }
    }
}

```

```

int motorNumber = input.substring(0, spaceIndex).toInt(); // Extract motor number
int stepsToMove = input.substring(spaceIndex + 1).toInt(); // Extract steps to move

// Validate motor number
if (motorNumber < 1 || motorNumber > 5) {
  Serial.println("Invalid motor number. Use 1, 2, 3, 4, or 5.");
  return;
}

// Call the corresponding motor's move function
switch (motorNumber) {
  case 1:
    moveMotor stepper1, stepsToMove, negativeStepsX,
MAX_NEGATIVE_STEPS);
    break;
  case 2:
    moveMotor stepper2, stepsToMove, negativeStepsY,
MAX_NEGATIVE_STEPS_MOTOR2);
    break;
  case 3:
    moveMotor3 stepper3, stepsToMove, negativeStepsZ,
MAX_NEGATIVE_STEPS_MOTOR3);
    break;
  case 4:
    moveMotor3 stepper4, stepsToMove, negativeStepsTilt,
MAX_NEGATIVE_STEPS_MOTOR4);
    //moveMotor stepper4, stepsToMove, negativeStepsTilt,
MAX_NEGATIVE_STEPS_MOTOR4);
    break;
  case 5:

```

```

        movePanMotor(stepper5, stepsToMove);
        break;
    }
}

void initializeMotors() {
    // Move motor1 in positive direction until limitSwitchX1 is activated
    stepper1.moveTo(1e6); // Move to a very large position
    while (true) {
        if (digitalRead(limitSwitchX1) == LOW) { // Switch is pressed
            stepper1.stop();           // Stop the motor
            stepper1.runToPosition();   // Ensure a clean stop
            stepper1.setCurrentPosition(0); // Set current position as 0
            break;                     // Exit the loop
        }
        stepper1.run(); // Continue moving
    }

    // Move motor2 in positive direction until limitSwitchY1 is activated
    stepper2.moveTo(1e6); // Move to a very large position
    while (true) {
        if (digitalRead(limitSwitchY1) == LOW) { // Switch is pressed
            stepper2.stop();
            stepper2.runToPosition();
            stepper2.setCurrentPosition(0);
            break;
        }
        stepper2.run();
    }
}

```

```

// Move motor3 in negative direction until limitSwitchZ1 is activated
stepper3.moveTo(-1e6); // Move to a very large negative position
while (true) {
    if (digitalRead(limitSwitchZ1) == LOW) { // Switch is pressed
        stepper3.stop();
        stepper3.runToPosition();
        stepper3.setCurrentPosition(0);
        break;
    }
    stepper3.run();
}

```

```

// Move motor4 in positive direction until limitSwitchTilt is activated
stepper4.moveTo(-1e6); // Move to a very large position
while (true) {
    if (digitalRead(limitSwitchTilt) == LOW) { // Switch is pressed
        stepper4.stop();
        stepper4.runToPosition();
        stepper4.setCurrentPosition(0);
        break;
    }
    stepper4.run();
}

```

```

// Move motor5 in positive direction until limitSwitchPan1 is activated
stepper5.moveTo(1e6); // Move to a very large position
while (true) {
    if (digitalRead(limitSwitchPan1) == LOW) { // Switch is pressed
        stepper5.stop();

```

```

    stepper5.runToPosition();
    break;
}
stepper5.run();
}

// Move motor5 back by MAX_STEPS_PAN / 2
stepper5.moveTo(stepper5.currentPosition() - (MAX_STEPS_PAN / 2));
while (stepper5.distanceToGo() != 0) {
    stepper5.run();
}
stepper5.setCurrentPosition(MAX_STEPS_PAN / 2);
}

void moveMotor(AccelStepper &stepper, int steps, int &negativeSteps, int
maxNegativeSteps) {
    // Check movement permissions based on negativeSteps and maxNegativeSteps
    if (steps < 0) {
        if (negativeSteps + abs(steps) > maxNegativeSteps) {
            Serial.println("Cannot move further in the negative direction. Input exceeds
Maximum limit.");
            return;
        }
        negativeSteps += abs(steps); // Update negative steps
    } else if (steps > 0) {
        if (negativeSteps - steps < 0) {
            Serial.println("Cannot move further in the positive direction. Input exceeds Position
zero.");
            return;
        }
        negativeSteps -= steps; // Update negative steps
    }
}

```

```

    } else {
        Serial.println("No movement specified.");
        return;
    }

    // Move the motor the specified number of steps
    stepper.moveTo(stepper.currentPosition() + steps);

    // Perform the motion
    while (stepper.distanceToGo() != 0) {
        stepper.run(); // Non-blocking motion
    }

    // Notify completion
    Serial.print("Moved ");
    Serial.print(steps);
    Serial.println(" steps.");
    Serial.print("Current negative steps: ");
    Serial.println(negativeSteps);
}

void movePanMotor(AccelStepper &stepper, int steps) {
    int targetPosition = stepper.currentPosition() + steps;

    // Ensure the target position is within bounds (0 to MAX_STEPS_PAN)
    if (targetPosition < 0 || targetPosition > MAX_STEPS_PAN) {
        Serial.println("Cannot move. Target position is out of bounds (0 to 1000).");
        return;
    }
}

```

```

// Move the motor the specified number of steps
stepper.moveTo(targetPosition);

// Perform the motion
while (stepper.distanceToGo() != 0) {
    stepper.run(); // Non-blocking motion
}

// Notify completion
Serial.print("Moved ");
Serial.print(steps);
Serial.println(" steps.");
Serial.print("Current position: ");
Serial.println(stepper.currentPosition());
}

// Add a new function for Motor 3
void moveMotor3(AccelStepper &stepper, int steps, int &negativeSteps, int
maxNegativeSteps) {
    // Check movement permissions based on negativeSteps and maxNegativeSteps
    if (steps > 0) {
        if (negativeSteps + steps > maxNegativeSteps) {
            Serial.println("Cannot move further in the Positive direction. Input exceeds
Maximum limit.");
            return;
        }
        negativeSteps += steps; // Update negative steps
    } else if (steps < 0) {
        if (negativeSteps - abs(steps) < 0) {
            Serial.println("Cannot move further in the Negative direction. Input exceeds
Position zero.");
        }
    }
}

```

```

        return;
    }
    negativeSteps -= abs(steps); // Update negative steps
} else {
    Serial.println("No movement specified.");
    return;
}

// Move the motor the specified number of steps
stepper.moveTo(stepper.currentPosition() + steps);

// Perform the motion
while (stepper.distanceToGo() != 0) {
    stepper.run(); // Non-blocking motion
}

// Notify completion
Serial.print("Moved ");
Serial.print(steps);
Serial.println(" steps.");
Serial.print("Current Position: ");
Serial.println(negativeSteps);
}

```

Processing-Code

```

import processing.serial.*;
import controlP5.*;
import java.awt.Font;

```

```

ControlP5 cp5;
Serial myPort; // Serial object
Textfield motorInput, stepsInput;
Button enterButton;
Label messageLabel;
String message = "Enter motor number and (Distance / Angle).";
int maxWidth = 260;
int xPos = 30;
int yPos = 360;
int lineSpacing = 20;
int[] maxStepsPerMotor = {25000, 5300, 2500, 800, 800};
int[] motorPositions = new int[5];
int MAX_STEPS_PAN = 800;
float radius = 75;
int indicatorXOffset = 140;
int labelOffsetX = 30;
int labelSize = 20;
int distanceBetweenIndicators = 110;
String noteMessage = "Note:\nPlease STOP and RUN the program\nagain, if you
rerun the Arduino\nprogram!";
int noteMaxWidth = 300;
int noteXPos = 20;
int noteYPos = 460;
int noteLineSpacing = 18;

void setup() {
    size(710, 560);
    cp5 = new ControlP5(this);

    // Initialize serial port

```

```
String portName = Serial.list()[0]; // Get the first available port  
myPort = new Serial(this, portName, 9600); // Set up the port at 9600 baud
```

```
Font font = new Font("Arial", Font.PLAIN, 20);  
PFont pfont = createFont(font.getName(), font.getSize());
```

```
motorInput = cp5.addTextfield("motorInput")  
    .setPosition(20, 120)  
    .setSize(220, 30)  
    .setFont(pfont)  
    .setLabel("");
```

```
stepsInput = cp5.addTextfield("stepsInput")  
    .setPosition(20, 200)  
    .setSize(220, 30)  
    .setFont(pfont)  
    .setLabel("");
```

```
enterButton = cp5.addButton("ENTER")  
    .setPosition(80, 280)  
    .setSize(100, 30)  
    .setFont(pfont)  
    .setColorBackground(color(0, 150, 0))  
    .setColorForeground(color(0, 0, 0))  
    .setColorActive(color(0, 0, 0))  
    .onClick(new CallbackListener() {  
        public void controlEvent(CallbackEvent event) {  
            sendMotorCommand();  
        }  
    });
```

```

// Initialize positions: Motors 1, 2, and 4 start at max, others start at 0 or mid-range
motorPositions[0] = maxStepsPerMotor[0]; // Motor 1 (X)
motorPositions[1] = maxStepsPerMotor[1]; // Motor 2 (Y)
motorPositions[2] = 0; // Motor 3 (Z)
motorPositions[3] = 0; // Motor 4 (Tilt)
motorPositions[4] = MAX_STEPS_PAN / 2; // Motor 5 (Pan)
}

void draw() {
  background(240);

  displayMessageWithWrap(noteMessage, noteXPos, noteYPos, noteMaxWidth,
color(255, 0, 0), 20);

  drawTextfieldLabels();

  fill(0, 0, 139);
  textSize(45);
  text("Camera Control", 20, 70);

  fill(255);
  noStroke();
  rect(20, 340, 270, 80);

  displayWrappedText(message, xPos, yPos, maxWidth);

  drawTextIndicator(); // Draw the text indicator

  float xPos = 350 + indicatorXOffset;
  float yPos = 100;

```

```

for (int i = 0; i < 5; i++) {
    drawMotorIndicator(i, xPos, yPos + i * distanceBetweenIndicators, radius,
motorPositions[i]);
}
}

```

```

void sendMotorCommand() {
    String motorNumberStr = motorInput.getText();
    String stepsStr = stepsInput.getText();

    if (motorNumberStr.isEmpty() || stepsStr.isEmpty()) {
        message = "Please enter both motor number and steps.";
        return;
    }
}

```

```

int motorNumber = int(motorNumberStr);
int steps = 0;

```

```

switch (motorNumber) {
    case 1:
        steps = round((float(stepsStr) / 190.0) * 25000);
        break;
    case 2:
        steps = round((float(stepsStr) / 200.0) * 5300);
        break;
    case 3:
        steps = round((float(stepsStr) / 72.0) * 2500);
        break;
    case 4:
        steps = round((float(stepsStr) / 60.0) * 800);

```

```

        break;
    case 5:
        steps = round((float(stepsStr) / 180.0) * 800);
        break;
}

```

```

if (motorNumber < 1 || motorNumber > 5) {
    message = "Invalid motor number. Please enter a number between 1 and 5.";
    return;
}

```

```

int motorIndex = motorNumber - 1;
int potentialPosition = motorPositions[motorIndex] + steps;

```

```

if (potentialPosition < 0) {
    switch (motorNumber) {
        case 1:
            message = "Error: Input exceeds zero position in X direction. Current X position: "
                + (int)((motorPositions[motorIndex] * 190.0 / 25000.0) + 0.5) + " cm.";
            break;
        case 2:
            message = "Error: Input exceeds zero position in Y direction. Current Y position: "
                + (int)((motorPositions[motorIndex] * 200.0 / 5300.0) + 0.5) + " cm.";
            break;
        case 3:
            message = "Error: Input exceeds zero position in Z direction. Current Z position: "
                + (int)((motorPositions[motorIndex] * 72.0 / 2500.0) + 0.5) + " cm.";
            break;
        case 4:
            message = "Error: Input exceeds zero position for Camera tilt. Current Tilt angle: "

```

```

        + (int)((motorPositions[motorIndex] * 60.0 / 800.0) + 0.5) + "°.";
    break;
case 5:
    message = "Error: Input exceeds zero position for Camera pan. Current Pan angle: "
    + (int)((motorPositions[motorIndex] * 180.0 / 800.0) + 0.5) + "°.";
    break;
}
return;
}

if (potentialPosition > maxStepsPerMotor[motorIndex]) {
    switch (motorNumber) {
        case 1:
            message = "Error: Input exceeds maximum position in X direction. Max position:
190 cm, Current X position: "
            + (int)((motorPositions[motorIndex] * 190.0 / 25000.0) + 0.5) + " cm.";
            break;
        case 2:
            message = "Error: Input exceeds maximum position in Y direction. Max position:
200 cm, Current Y position: "
            + (int)((motorPositions[motorIndex] * 200.0 / 5300.0) + 0.5) + " cm.";
            break;
        case 3:
            message = "Error: Input exceeds maximum position in Z direction. Max position:
72 cm, Current Z position: "
            + (int)((motorPositions[motorIndex] * 72.0 / 2500.0) + 0.5) + " cm.";
            break;
        case 4:
            message = "Error: Input exceeds maximum position for Camera tilt. Max position:
60°, Current Tilt angle: "

```

```

        + (int)((motorPositions[motorIndex] * 60.0 / 800.0) + 0.5) + "°.";
    break;

    case 5:
        message = "Error: Input exceeds maximum position for Camera pan. Max
position: 180°, Current Pan angle: "
            + (int)((motorPositions[motorIndex] * 180.0 / 800.0) + 0.5) + "°.";
        break;
    }
    return;
}

motorPositions[motorIndex] = potentialPosition;

String command = motorNumber + " " + steps;
myPort.write(command + "\n"); // Send the command to Arduino

// switch (motorNumber) {
// case 1:
//     message = "Camera moved in X direction. Current X position: "
//         + round(motorPositions[motorIndex] * 190 / 25000) + " cm";
//     break;
// case 2:
//     message = "Camera moved in Y direction. Current Y position: "
//         + round(motorPositions[motorIndex] * 200 / 5300) + " cm";
//     break;
// case 3:
//     message = "Camera moved in Z direction. Current Z position: "
//         + round(motorPositions[motorIndex] * 72 / 2500) + " cm";
//     break;
// case 4:

```

```

// message = "Camera tilted. Current Tilt angle: "
//      + round(motorPositions[motorIndex] * 60 / 800) + "°";
// break;
// case 5:
// message = "Camera panned. Current Pan angle: "
//      + round(motorPositions[motorIndex] * 180 / 800) + "°";
// break;
//}

```

```

switch (motorNumber) {
case 1:
    message = "Camera moved in X direction. Current X position: "
        + (int)((motorPositions[motorIndex] * 190.0 / 25000.0) + 0.5) + " cm";
    break;
case 2:
    message = "Camera moved in Y direction. Current Y position: "
        + (int)((motorPositions[motorIndex] * 200.0 / 5300.0) + 0.5) + " cm";
    break;
case 3:
    message = "Camera moved in Z direction. Current Z position: "
        + (int)((motorPositions[motorIndex] * 72.0 / 2500.0) + 0.5) + " cm";
    break;
case 4:
    message = "Camera tilted. Current Tilt angle: "
        + (int)((motorPositions[motorIndex] * 60.0 / 800.0) + 0.5) + "°";
    break;
case 5:
    message = "Camera panned. Current Pan angle: "
        + (int)((motorPositions[motorIndex] * 180.0 / 800.0) + 0.5) + "°";
    break;
}

```

```
}
```

```
motorInput.clear();
```

```
stepsInput.clear();
```

```
}
```

```
void displayWrappedText(String msg, float x, float y, float maxWidth) {
```

```
  fill(0);
```

```
  textSize(16);
```

```
  String[] words = split(msg, " ");
```

```
  String currentLine = "";
```

```
  for (int i = 0; i < words.length; i++) {
```

```
    if (textWidth(currentLine + words[i]) < maxWidth) {
```

```
      currentLine += words[i] + " ";
```

```
    } else {
```

```
      text(currentLine, x, y);
```

```
      y += lineSpacing;
```

```
      currentLine = words[i] + " ";
```

```
    }
```

```
  }
```

```
  text(currentLine, x, y);
```

```
}
```

```
void drawMotorIndicator(int motorIndex, float x, float y, float radius, float position) {
```

```
  float angle = map(position, 0, maxStepsPerMotor[motorIndex], -PI, 0);
```

```
  noFill();
```

```
stroke(0);
strokeWeight(2);
for (int i = 0; i < radius; i++) {
    float alpha = map(i, 0, radius, 100, 255);
    stroke(0, 191, 255, alpha);
    arc(x, y, (i * 2), (i * 2), -PI, 0, OPEN);
}
```

```
stroke(0);
strokeWeight(3);
arc(x, y, radius * 2, radius * 2, -PI, 0, OPEN);
```

```
pushMatrix();
translate(x, y);
rotate(angle);
stroke(255, 0, 0);
strokeWeight(6);
line(0, 0, radius, 0);
popMatrix();
```

```
fill(0);
textSize(labelSize);
```

```
String minLabel = "";
String maxLabel = "";
switch (motorIndex) {
    case 0: minLabel = "min X"; maxLabel = "max X"; break;
    case 1: minLabel = "min Y"; maxLabel = "max Y"; break;
    case 2: minLabel = "min Z"; maxLabel = "max Z"; break;
    case 3: minLabel = "min Tilt"; maxLabel = "max Tilt"; break;
```

```

    case 4: minLabel = "min Pan"; maxLabel = "max Pan"; break;
}

textAlign(RIGHT);
text(minLabel, x - radius - 10, y);
textAlign(LEFT);
text(maxLabel, x + radius + 10, y);
}

void drawTextfieldLabels() {
    textSize(16);
    fill(0);

    text("Motor Number", 20, 170);
    text("(Distance / Angle) to Move", 20, 250);
}

void displayMessageWithWrap(String msg, float x, float y, float maxWidth, color
textColor, int textSize) {
    fill(textColor);
    textSize(textSize);

    String[] words = split(msg, " ");
    String currentLine = "";

    for (int i = 0; i < words.length; i++) {
        if (textWidth(currentLine + words[i]) < maxWidth) {
            currentLine += words[i] + " ";
        } else {
            text(currentLine, x, y);

```

```

    y += noteLineSpacing;
    currentLine = words[i] + " ";
}
}

text(currentLine, x, y);
}

void drawTextIndicator() {
    fill(0); // Set the text color to black
    textSize(16); // Set the text size to 16
    text("0 cm", 373, 80); // Example: Draw the text "0" at position (50, 50)

    fill(0); // Set the text color to black
    textSize(16); // Set the text size to 16
    text("0 cm", 373, 190); // Example: Draw the text "0" at position (50, 50)

    fill(0); // Set the text color to black
    textSize(16); // Set the text size to 16
    text("0 cm", 373, 300); // Example: Draw the text "0" at position (50, 50)

    fill(0); // Set the text color to black
    textSize(16); // Set the text size to 16
    text("0°", 390, 410); // Example: Draw the text "0" at position (50, 50)

    fill(0); // Set the text color to black
    textSize(16); // Set the text size to 16
    text("0°", 390, 520); // Example: Draw the text "0" at position (50, 50)

    fill(0); // Set the text color to black

```

```
textSize(16); // Set the text size to 16
text("190 cm", 575, 80); // Example: Draw the text "0" at position (50, 50)

fill(0); // Set the text color to black
textSize(16); // Set the text size to 16
text("200 cm", 575, 190); // Example: Draw the text "0" at position (50, 50)

fill(0); // Set the text color to black
textSize(16); // Set the text size to 16
text("72 cm", 575, 300); // Example: Draw the text "0" at position (50, 50)

fill(0); // Set the text color to black
textSize(16); // Set the text size to 16
text("60°", 575, 410); // Example: Draw the text "0" at position (50, 50)

fill(0); // Set the text color to black
textSize(16); // Set the text size to 16
text("180°", 575, 520); // Example: Draw the text "0" at position (50, 50)
}
```

Datenvorbereitung

```
clear all
```

```
close all
```

```
clc
```

```
load("gTruth.mat")
```

```
imds = imageDatastore(gTruth.imageFilename);
```

```
blds = boxLabelDatastore(gTruth(:,2));
```

```
imageFile = 'IMG_1.JPG';
```

```
boxes = blds.LabelData{10};
```

```
labels = blds.LabelData{10, 2};
```

```
Bild = imread(imageFile);
```

```
Masks = poly2mask(gTruth.pol{10,1}(:,1),  
gTruth.pol{10,1}(:,2),size(Bild,1),size(Bild,2)); %Originalbildgröße
```

Mask R-CNN for Instance Segmentation

```
clear all

close all

clc

dataDir = 'C:\Users\Ismail Tayoub\Desktop\matlab';

annsDir = fullfile(dataDir,'train_seg_1');

ds =
fileDatastore(annsDir,FileExtensions=".mat",ReadFcn=@(x)matReaderBinData(x,dataDir));

net = maskrcnn("resnet50-coco","rec");

options = trainingOptions("sgdm", ...
    InitialLearnRate=0.001, ...
    LearnRateSchedule="piecewise", ...
    LearnRateDropPeriod=1, ...
    LearnRateDropFactor=0.95, ...
    Plot="none", ...
    Momentum=0.9, ...
    MaxEpochs=10, ...
    MiniBatchSize=2, ...
    BatchNormalizationStatistics="moving", ...
    ResetInputNormalization=false, ...
    ExecutionEnvironment="auto", ...
    VerboseFrequency=50);
```

```
[net,info] = trainMaskRCNN(ds,net,options,FreezeSubNetwork="backbone");  
  
I = imread('IMG_1.jpg');  
  
[masks,labels,scores,boxes] = segmentObjects(net,I,Threshold=0.5);  
  
overlayedImage = insertObjectMask(I,masks);  
  
imshow(overlayedImage)
```

Visual SLAM with RGB-D Camera

```
clear
```

```
close all
```

```
clc
```

```
I1=imread('IMG_14.jpg');
```

```
I2=imread('IMG_12.jpg');
```

```
figure
```

```
imshowpair(I1, I2, 'montage');
```

```
title('Original Images');
```

```
load('calibrationSession.mat')
```

```
intrinsics = calibrationSession.CameraParameters.Intrinsics;
```

```
I2_resized = imresize(I2, intrinsics.ImageSize);
```

```
I2_undistorted = undistortImage(I2_resized, intrinsics);
```

```
I1 = undistortImage(I1, intrinsics);
```

```
I2 = undistortImage(I2, intrinsics);
```

```
figure
```

```
imshowpair(I1, I2, "montage");
```

```
title("Undistorted Images");
```

```
% Detect feature points
```

```
imagePoints1 = detectMinEigenFeatures(im2gray(I1), MinQuality=0.01);
```

```

% Visualize detected points

figure

imshow(I1, InitialMagnification = 50);

title("150 Strongest Corners from the First Image");

hold on

plot(selectStrongest(imagePoints1, 150));


% Create the point tracker

tracker = vision.PointTracker(MaxBidirectionalError=1, NumPyramidLevels=5);


% Initialize the point tracker

imagePoints1 = imagePoints1.Location;

initialize(tracker, imagePoints1, I1);


% Track the points

[imagePoints2, validIdx] = step(tracker, I2);

matchedPoints1 = imagePoints1(validIdx, :);

matchedPoints2 = imagePoints2(validIdx, :);


% Visualize correspondences

figure

showMatchedFeatures(I1, I2, matchedPoints1, matchedPoints2);

title("Tracked Features");

```

```

% Estimate the fundamental matrix

[E, epipolarInliers] = estimateEssentialMatrix(...
    matchedPoints1, matchedPoints2, intrinsics, Confidence = 99.99);

% Find epipolar inliers

inlierPoints1 = matchedPoints1(epipolarInliers, :);
inlierPoints2 = matchedPoints2(epipolarInliers, :);

% Display inlier matches

figure

showMatchedFeatures(I1, I2, inlierPoints1, inlierPoints2);

title("Epipolar Inliers");

relPose = estrelpose(E, intrinsics, inlierPoints1, inlierPoints2);

% Detect dense feature points. Use an ROI to exclude points close to the
% image edges.

border = 30;

roi = [border, border, size(I1, 2)- 2*border, size(I1, 1)- 2*border];

imagePoints1 = detectMinEigenFeatures(im2gray(I1), ROI = roi, ...
    MinQuality = 0.001);

% Create the point tracker

tracker = vision.PointTracker(MaxBidirectionalError=1, NumPyramidLevels=5);

```

```

% Initialize the point tracker

imagePoints1 = imagePoints1.Location;

initialize(tracker, imagePoints1, I1);


% Track the points

[imagePoints2, validIdx] = step(tracker, I2);

matchedPoints1 = imagePoints1(validIdx, :);
matchedPoints2 = imagePoints2(validIdx, :);


% Compute the camera matrices for each position of the camera

% The first camera is at the origin looking along the Z-axis. Thus, its
% transformation is identity.

camMatrix1 = cameraProjection(intrinsics, rigidtfom3d);
camMatrix2 = cameraProjection(intrinsics, pose2extr(relPose));


% Compute the 3-D points

points3D = triangulate(matchedPoints1, matchedPoints2, camMatrix1, camMatrix2);


% Get the color of each reconstructed point

numPixels = size(I1, 1) * size(I1, 2);

allColors = reshape(I1, [numPixels, 3]);

colorIdx = sub2ind([size(I1, 1), size(I1, 2)], round(matchedPoints1(:,2)), ...
    round(matchedPoints1(:, 1)));

color = allColors(colorIdx, :);

```

```

% Create the point cloud

ptCloud = pointCloud(points3D, Color=color);

% Visualize the camera locations and orientations

cameraSize = 0.3;

figure

plotCamera(Size=cameraSize, Color="r", Label="1", Opacity=0);

hold on

grid on

plotCamera(AbsolutePose=relPose, Size=cameraSize, ...
    Color="b", Label="2", Opacity=0);

% Visualize the point cloud

pcshow(ptCloud, VerticalAxis="y", VerticalAxisDir="down", MarkerSize=45);

% Rotate and zoom the plot

camorbit(0, -30);

camzoom(1.5);

% Label the axes

xlabel("x-axis");

ylabel("y-axis");

zlabel("z-axis")

title("Up to Scale Reconstruction of the Scene")

```

